arXiv:2405.12981v1 [cs.LG] 21 May 2024

Reducing Transformer Key-Value Cache Size with Cross-Layer Attention

William Brandon[*]
MIT CSAIL
wbrandon@csail.mit.edu

Mayank Mishra[*]
MIT-IBM Watson AI Lab

Aniruddha Nrusimha
MIT CSAIL

Rameswar Panda
MIT-IBM Watson AI Lab

Jonathan Ragan-Kelley
MIT CSAIL

Abstract

Key-value (KV) caching plays an essential role in accelerating decoding for transformer-based autoregressive large language models (LLMs). However, the amount of memory required to store the KV cache can become prohibitive at long sequence lengths and large batch sizes. Since the invention of the transformer, two of the most effective interventions discovered for reducing the size of the KV cache have been Multi-Query Attention (MQA) and its generalization, Grouped-Query Attention (GQA). MQA and GQA both modify the design of the attention block so that multiple query heads can share a single key/value head, reducing the number of distinct key/value heads by a large factor while only minimally degrading accuracy. In this paper, we show that it is possible to take Multi-Query Attention a step further by also sharing key and value heads between adjacent layers, yielding a new attention design we call Cross-Layer Attention (CLA). With CLA, we find that it is possible to reduce the size of the KV cache by another $2\times$ while maintaining nearly the same accuracy as unmodified MQA. In experiments training 1B- and 3B-parameter models from scratch, we demonstrate that CLA provides a Pareto improvement over the memory/accuracy tradeoffs which are possible with traditional MQA, enabling inference with longer sequence lengths and larger batch sizes than would otherwise be possible.

1 Introduction

The memory footprint of the key-value (KV) cache can be a bottleneck when serving large language models (LLMs). Because the size of the KV cache scales proportionally with both sequence length and batch size, the memory overhead of KV cache storage can limit batch sizes when operating on long sequence lengths [Chowdhery et al., 2022], and can require employing costly techniques like offloading when on-device memory is scarce [Sheng et al., 2023]. It is also desirable to be able to persist KV caches over long periods of time in order to minimize redundant computations [Gao et al., 2024, Google, 2024]. However, the size of the KV cache directly determines the cost of storing and retrieving such persistent caches. As new applications of LLMs emerge which demand ever-longer sequence lengths, the memory footprint of the KV cache is becoming an increasingly important consideration in the design of efficient transformer-based language models.

Existing work has proposed a variety of methods for decreasing the memory footprint of the KV cache, including storing KV activations in low precision [Hooper et al., 2024, Zhang et al., 2024],

[*]Equal Contribution

Preprint. Under review.

evicting unimportant KV cache entries [Zhang et al., 2023, Liu et al., 2023], and sharing keys and values across query heads in the attention mechanism [Shazeer, 2019, Ainslie et al., 2023b].

In this paper, we introduce a method for reducing the size of the KV cache along a dimension different than those explored in prior work: namely, reducing the number of unique *layers* in the KV cache. Our contributions are as follows:

1. We propose *Cross-Layer Attention* (CLA), a modification to the transformer architecture which reduces the size of the KV cache by *sharing KV activations across layers*.
2. We conduct extensive pretraining experiments to characterize the effect of different CLA configurations on accuracy and memory usage across a range of architectural hyperparameters, learning rates and model sizes.
3. We demonstrate that CLA enables accuracy/memory Pareto improvements relative to existing Multi-Query Attention (MQA) and Grouped-Query Attention (GQA) architectures.
4. In particular, we demonstrate at the 1B- and 3B-parameter scales that combining CLA with MQA can achieve a $2\times$ reduction in KV cache size versus a plain MQA baseline, with minimal degradation in perplexity.
5. We offer guidance on which CLA configurations perform best based on our experiments, finding that CLA should be used between pairs of consecutive layers, and that CLA appears to deliver the most robust benefits when used in conjunction with MQA.

2 Cross-Layer Attention

In this section we describe our Cross-Layer Attention (CLA) technique, and its relationship to the KV-sharing mechanisms employed by the existing Multi-Query and Grouped-Query attention architectures (MQA and GQA).

2.1 Background: Multi-Query Attention and Grouped-Query Attention

The original transformer architecture employed Multi-Head Attention (MHA) [Vaswani et al., 2017], in which each query head attends over the keys and values produced by a distinct key/value head. In MHA, the KV activations of each key/value head must be stored separately in the KV cache, resulting in a storage overhead of $2 \cdot n_{\text{query}} \cdot d_{\text{head}}$ elements per token, where n_{query} is the number of query heads and d_{head} is the embedding dimension of each head.

To reduce the overhead associated with storing and accessing the KV cache during transformer decoding, Shazeer [2019] proposed *Multi-Query Attention* (MQA), which Ainslie et al. later generalized to *Grouped-Query Attention* (GQA). Grouped-Query Attention modifies the transformer architecture by organizing the query heads of each attention layer into groups, where each group of query heads shares a single key/value head. Because the size of the KV cache scales only with the number of distinct key/value heads, not the number of query heads, GQA reduces the storage overhead of the KV cache to $2 \cdot n_{\text{group}} \cdot d_{\text{head}}$, where n_{group} denotes the number of groups for GQA and $n_{\text{group}} < n_{\text{query}}$. MQA can be seen as the special case of GQA in which $n_{\text{group}} = 1$.

Shazeer and Ainslie et al. find that MQA and GQA enable significant reductions in KV cache size and decoding latency while incurring only a small degradation in accuracy compared to MHA architectures with the same head dimension. The family of attention architectures enabled by using MQA and GQA defines an *accuracy/memory tradeoff space* in which model designers can choose how they want to balance the expressive power and KV cache overhead of their attention mechanism. MQA and GQA stake out different positions in this tradeoff space, and neither is necessarily preferable to the other for all use cases.

2.2 Sharing KV Activations Across Layers

Inspired by the success of MQA and GQA, which share key/value heads across query heads within a single layer, we propose also sharing key/value heads *across layers*. We refer to such an attention architecture as *Cross-Layer Attention* (CLA), and present a diagrammatic view of it in Figure 1. CLA computes key/value projections for only a subset of layers in the model; the attention blocks in layers without key/value projections reuse the KV activations of previous layers. Only the subset of layers

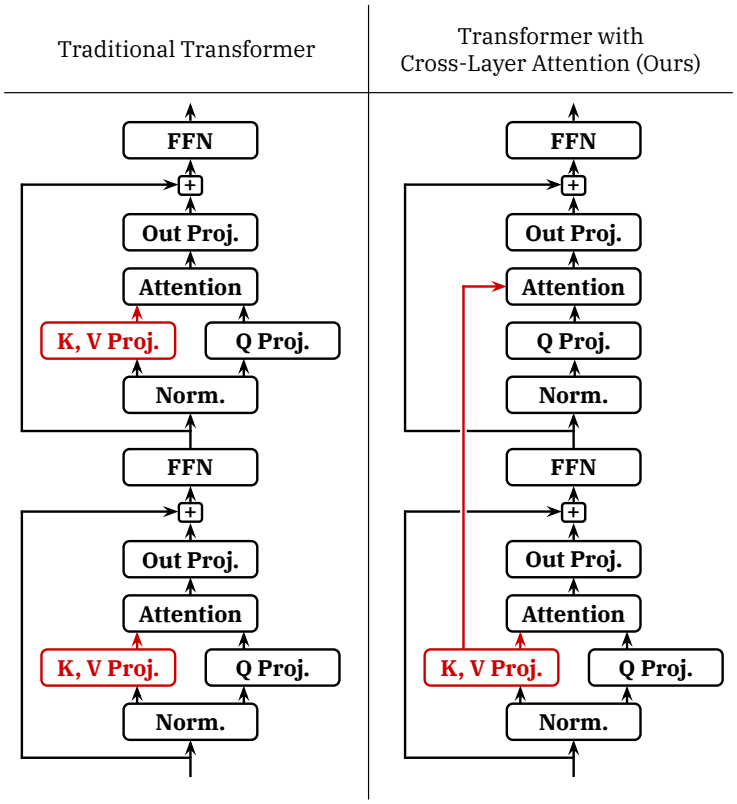

Figure 1: Schematic of two consecutive layers in a transformer using a traditional attention design (left) and in a transformer using Cross-Layer Attention (right). When using traditional attention, each layer computes its own separate K and V activations, which must be cached on a per-layer basis during autoregressive decoding. When using Cross-Layer Attention, some layers compute their own fresh K and V activations, while other layers reuse the K and V activations of earlier layers.

with key/value projections contribute to the KV cache, allowing a reduction in memory footprint relative to traditional architectures which apply a separate key/value projection in each layer.

CLA is orthogonal to MQA/GQA/MHA, and can be combined with any of them. Moreover, in the same way that GQA allows varying n_{group} to access a family of different attention configurations, CLA allows varying the number of layers which share the output of each KV projection, which we refer to as the *sharing factor*. We refer to different configurations of CLA by their sharing factors, giving rise to CLA2, which shares each KV projection among a pair of adjacent layers, CLA3, which shares each KV projection among a group of 3 layers, and so on. We present a visualization of different attention configurations possible with CLA in Figure 2.

2.3 Implications for System Design

CLA is primarily an intervention to reduce the memory footprint of the KV cache, and only has minor effects on other resources consumed by the model during training and inference. Here, we summarize the effect of CLA on key metrics relevant from a systems engineering perspective, assuming all other architectural hyperparameters are held constant:

- **KV Cache Memory**: CLA significantly reduces KV cache memory footprint, shrinking it by a factor equal to the sharing factor, or slightly less if the sharing factor does not evenly divide the number of layers.
- **Training Memory Footprint**: CLA reduces the memory footprint of intermediate KV activation tensors materialized during training, although for GQA and MQA models such KV tensors are typically small compared to the model's hidden states and MLP activations.

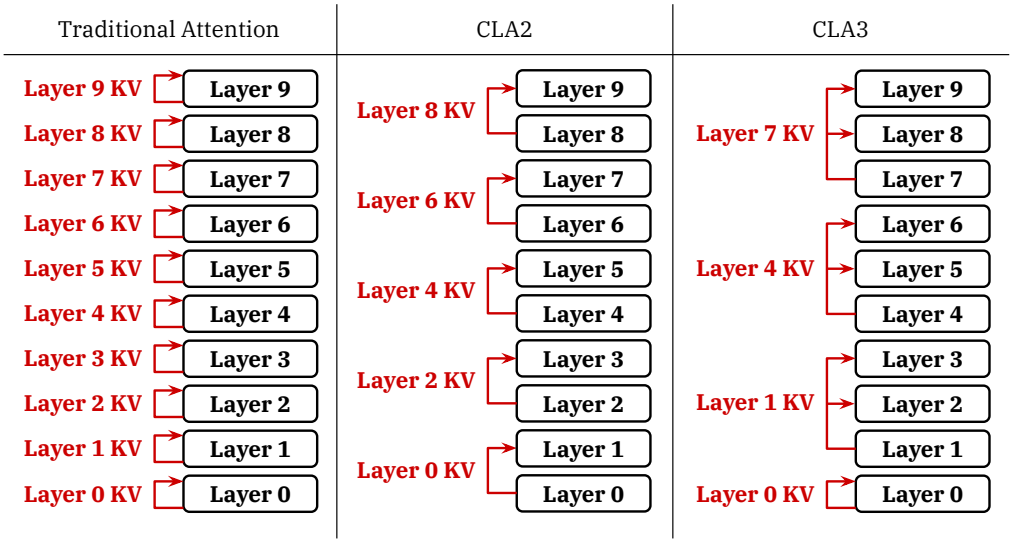

Figure 2: Schematic of KV cache structures under different attention configurations in a 10-layer transformer. Using traditional attention, each layer has its own KV cache. Using Cross-Layer Attention with a sharing factor of 2 (CLA2), every group of 2 consecutive layers shares a single KV cache. Using Cross-Layer Attention with a sharing factor of 3 (CLA3), every group of 3 consecutive layers shares a single KV cache. When the sharing factor does not evenly divide the number of layers, as in the CLA3 example, some KV caches must be shared over fewer layers than others; in this CLA3 configuration, we arbitrarily select the layer 0 KV cache to be used only in layer 0.

- **Model Parallelism**: CLA is fully compatible with standard tensor parallelism techniques [Shoeybi et al., 2020] for sharding model weights across multiple accelerators. In the presence of pipeline parallelism [Huang et al., 2019], either different layers which share a KV cache must be kept in the same pipeline stage, or else KV activations must be communicated between pipeline stages.
- **Parameters and FLOPs**: Because CLA reduces the total number of key/value projection blocks in the model, CLA slightly reduces the number of parameters in the model and the number of FLOPs required during a forward or backward pass.
- **Decoding Latency**: In the context of a full LLM serving stack, CLA can enable larger batch sizes and longer KV cache persistence times than would otherwise be possible, which have the potential to improve inference latency.
- **Core Attention Latency**: Unlike MQA and GQA, CLA has no direct effect on the memory bandwidth consumed by the attention mechanism in each decoding step, because even shared KV cache layers must be separately re-read from main memory in each attention layer. CLA therefore has no direct effect on the latency of the core attention computation during decoding.

3 Pretraining Experiments

To determine the effect of Cross-Layer Attention on language modeling accuracy, we trained a collection of transformer-based language models from scratch at the 1 billion and 3 billion parameter scales. While running these experiments, we sought to answer the following questions:

1. What accuracy/memory tradeoffs are possible using CLA?
2. How does using CLA compare to using plain GQA or MQA?
3. How does CLA interact with GQA and MQA?
4. What CLA configurations perform best given a fixed memory budget?
5. Are the effects of CLA consistent across scales?

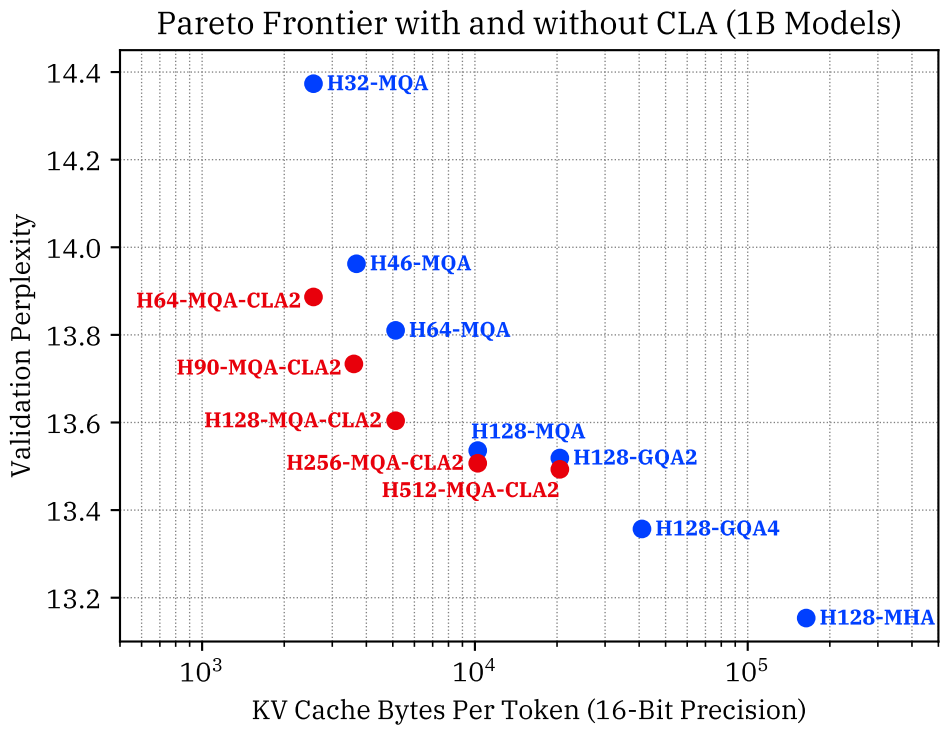

Figure 3: The accuracy/memory Pareto frontier discovered in our 1B-scale design space exploration, for models with CLA (red) and without CLA (blue). Lower is better on both axes.

We found that CLA enables favorable accuracy/memory tradeoffs compared to what is possible using plain GQA or MQA. Moreover, we found that in our experimental regime, a sharing factor of 2 is more effective than other sharing factors, and that CLA is consistently effective when combined with MQA when trying to decrease KV cache storage. We also found preliminary evidence to suggest that CLA models benefit from training with higher learning rates than comparable non-CLA models. Finally, we found that CLA confers benefits at both 1B- and 3B-parameter scales. In the rest of this section, we present our experimental setup and results in more detail.

3.1 Common Experimental Parameters

In all our experiments, we train our models from scratch on data from the SlimPajama [Soboleva et al., 2023] dataset, tokenized with the GPT-NeoX tokenizer [Black et al., 2022] which uses Byte-Pair Encoding (BPE) [Wang et al., 2019]. We adopt a Llama-like [Touvron et al., 2023] architecture with pre-normalization, SwiGLU activations [Shazeer, 2020, Ramachandran et al., 2017], and rotary position embeddings [Su et al., 2023]. We do not use dropout for any of our models. Our models include learnable elementwise affine parameters for layer-norm, and our CLA models use separately-learnable affine layer-norm parameters for the KV projection blocks and Q projection blocks in attention. Unless otherwise stated, we always set the number of query heads n_{query} such that $n_{\text{query}} \cdot d_{\text{head}}$ is equal to the hidden size d_{model}.

We train all models using the AdamW optimizer [Loshchilov and Hutter, 2019] with gradient clipping, using $\beta_1 = 0.9$, $\beta_2 = 0.95$, a weight decay factor of 0.1, and a clipping norm of 1.0. We use a linear learning rate warmup for the first 5% of training examples and a cosine learning rate schedule Loshchilov and Hutter [2017] decaying to 10% of the peak learning rate over the remainder of training. We set the sequence length to 2048 tokens and the batch size to 2048 sequences, for a total of $\approx 4M$ tokens per training step. All our experiments initialize the weights of linear layers from a normal distribution with mean zero and standard deviation 0.01275.

Model	d_{head}	Query Heads	KV Heads	KV Layers	KV Bytes Per Token (16-Bit)	Validation Perplexity
Non-CLA Baselines						
H128-MHA	128	16	16	20	163 840	13.15
H128-GQA4	128	16	4	20	40 960	13.36
H128-GQA2	128	16	2	20	20 480	13.52
H128-MQA	128	16	1	20	10 240	13.54
H64-MQA	64	32	1	20	5120	13.81
H46-MQA	46	45	1	20	3680	13.96
H32-MQA	32	64	1	20	2560	14.37
MQA + CLA2 Models						
H512-MQA-CLA2	512	4	1	10	20 480	13.49
H256-MQA-CLA2	256	8	1	10	10 240	13.51
H128-MQA-CLA2	128	16	1	10	5120	13.60
H90-MQA-CLA2	90	22	1	10	3600	13.73
H64-MQA-CLA2	64	32	1	10	2560	13.89
GQA + CLA2 Models						
H256-GQA4-CLA2	256	8	4	10	40 960	13.38
H128-GQA4-CLA2	128	16	4	10	20 480	13.48
H128-GQA2-CLA2	128	16	2	10	10 240	13.59
MQA + CLA $>$ 2 Models						
H128-MQA-CLA3	128	16	1	7	3584	13.77
H128-MQA-CLA4	128	16	1	5	2560	13.95
MQA + CLA2, Non-Uniform Sharing						
H128-MQA-CLA2-KeepEnds	128	16	1	11	5632	13.62
H128-MQA-CLA2-DenseFront	128	16	1	11	5632	13.75
H128-MQA-CLA2-DenseBack	128	16	1	11	5632	14.03

Table 1: Results of our 1B-scale design space exploration.

Model Family	Hidden Size	FFN Size	Layers	Sequence Length	Training Tokens
1B Models	2048	5472	20	2048	30×10^9
3B Models	3072	8192	32	2048	100×10^9

Table 2: Architectural and training hyperparameters shared across our pretraining experiments.

We perform all experiments on NVIDIA H100 GPUs using PyTorch [Paszke et al., 2019, Ansel et al., 2024]. We use mixed precision training [Micikevicius et al., 2018] in BF16 [Kalamkar et al., 2019] with gradient all-reduce and gradient accumulation in FP32 for training stability.

3.2 Experiments at 1B-Parameter Scale

We trained all our 1B-scale models on 30 billion tokens using a consistent data order, and, other than varying the attention mechanism, used the same architectural hyperparameters across all 1B-scale models. This means that all our 1B models were all trained using approximately the same number of FLOPs and approximately the same number of GPU-hours, with CLA models requiring slightly fewer FLOPs to train than their non-CLA counterparts due to the reduced number of key/value projections. The common hyperparameters shared across our 1B-scale experiments can be found in Table 2.

We ran two main sets of experiments at the 1B-parameter scale. First, we trained a diverse set of CLA and non-CLA models to characterize the range of accuracy/memory tradeoffs achievable with and without CLA, and to determine which CLA configurations are most effective; we refer to these as our *design space exploration* experiments, and describe them in more detail in Section 3.2.1. Second, we

conducted a learning rate sweep on a subset of models from our design space exploration to verify that our results continue to hold even against a strong non-CLA baseline with a well-tuned learning rate. We describe these learning rate tuning experiments in Appendix A.

3.2.1 Design Space Exploration

The primary goal of our 1B-parameter-scale design space exploration was to characterize the Pareto frontier of accuracy/memory tradeoffs achievable with and without CLA, and to determine which CLA configurations achieve the best accuracy on a fixed KV cache memory budget. We train all models in our design space exploration using a learning rate of $\text{LR} = 3 \times 10^{-4}$, which we determined to be conservative; we explore the effect of the learning rate on accuracy in more detail in Section 3.2.2.

For our design space exploration, we first trained a collection of seven non-CLA baseline models along the MHA-GQA-MQA spectrum, exhibiting a range of KV cache memory requirements spanning two orders of magnitude. Our baseline model with the largest KV cache memory footprint is an MHA model with a head embedding dimension of $d_{\text{head}} = 128$ (163840 bytes per token at 16-bit precision), and our baseline with the smallest footprint is an MQA model with head dimension $d_{\text{head}} = 32$ (2560 bytes per token).

We quantify the accuracy of models in our design space exploration using perplexity on a held-out validation set of $\approx 4M$ tokens drawn from our SlimPajama corpus. A summary of results for the models in our design space exploration, including our baseline models, can be found in Table 1. We adopt the naming scheme "H$\langle d_{\text{head}} \rangle$-$\langle$attention mechanism$\rangle$" for all models in our experiments, so that, for example, a model employing MQA with a head dimension of $d_{\text{head}} = 64$ would be named "H64-MQA." For our baseline models, we observe that validation perplexity increases monotonically as we reduce the memory capacity of the KV cache, ranging from a perplexity of 13.15 for our H128-MHA baseline to 14.37 for our H32-MQA baseline.

In the rest of this section, we present results for the CLA models we trained during our design space exploration.

Best Performance: MQA + CLA2. We trained a family of five models combining MQA with CLA2. We varied the head dimension for our MQA-CLA2 models from $d_{\text{head}} = 512$ down to $d_{\text{head}} = 64$, allowing us to compare to a range of non-CLA baseline models with varying KV cache capacities.

We found that our MQA-CLA2 models are able to achieve better perplexities than baseline models requiring the same amount of KV cache memory, advancing the accuracy/memory Pareto frontier. We present a plot of the accuracy/memory Pareto frontier with and without CLA in Figure 3. Our MQA-CLA2 models with head dimensions $d_{\text{head}} \in \{64, 90, 128\}$ are able to match the KV cache memory footprint of baseline MQA models with head dimensions $d_{\text{head}} \in \{32, 46, 64\}$ while achieving substantial perplexity improvements in the range of 0.21–0.48 points. Additionally, our MQA-CLA2 models with large head sizes of $d_{\text{head}} \in \{256, 512\}$ are able to match the KV cache footprint of our MQA and GQA2 baselines with $d_{\text{head}} = 128$ while achieving a small perplexity improvement of 0.03 points.

We found that our MQA-CLA2 models achieved the best accuracy/memory tradeoffs among all CLA configurations we tested in our design space exploration. In the rest of this section, we briefly describe the ablations we conducted to explore alternate CLA configurations.

Ablation: GQA + CLA2. We trained three models to explore combining GQA with CLA2. We chose GQA4-CLA2 with $d_{\text{head}} = 128$ as our starting point, as GQA4 represents an attention configuration intermediate between our MQA and MHA baselines. We then explored expanding the head dimension of our GQA4-CLA2 model to $d_{\text{head}} = 256$, as well as reducing the GQA factor to GQA2. We found that only the GQA2-CLA2 configuration was able to achieve a perplexity better than the corresponding baseline model with the same KV cache footprint, and that this perplexity was the same (within 0.01 points) as our MQA-CLA2 model with the same footprint.

Ablation: MQA + CLA with Sharing Factor > 2. To explore the effect of using CLA sharing factors > 2, we trained MQA-CLA3 and MQA-CLA4 models with head dimension $d_{\text{head}} = 128$. We found that these CLA3 and CLA4 models achieved a Pareto improvement over our plain MQA

baselines, matching the KV cache footprint of our baseline MQA models with head dimensions of $d_{\text{head}} \in \{32, 46\}$ while achieving better perplexities. However, we found that they achieved worse perplexities than our MQA-CLA2 models at the same KV cache footprint.

Ablation: MQA + CLA2 with Non-Uniform Sharing Patterns. Finally, we explored using different patterns of KV activation sharing in our MQA-CLA2 models.

On the hypothesis that the first and last layers in the model might benefit from special treatment, we trained a model "H128-MQA-CLA2-KeepEnds" which does not share the layer 0 KV cache with any other layers, and instead groups layer 1 with layer 2, groups layer 3 with layer 4, and so on. This also has the effect of giving the final layer its own KV cache separate from all other layers.

We also explored imbalanced configurations with all the KV-cache-producing layers concentrated at either the beginning or end of the model. We trained a model "H128-MQA-CLA2-DenseFront" consisting of 10 non-CLA layers, followed by 9 CLA layers all using the KV activations of layer 9, and a final layer with its own KV cache. Similarly, we trained a model "H128-MQA-CLA2-DenseBack" consisting of 2 non-CLA layers, followed by a run of 10 CLA layers all using the KV activations of layer 1, and finally 9 non-CLA layers.

We found that all of these alternative CLA sharing patterns achieve worse perplexities than the corresponding MQA-CLA2 model with a uniform sharing pattern, while also requiring slightly more KV cache memory.

3.2.2 Robustness to Learning Rate Tuning

The relative performance of different model architectures can change depending on the learning rates at which they are evaluated. To account for the effects of the learning rate on our results, we conducted learning rate tuning experiments on three models of interest from our initial 1B-scale design space exploration. These learning rate tuning experiments help us verify that CLA continues to provide benefits even when compared to baselines trained at their optimal learning rates.

We chose to tune the learning rate for the baseline models H128-MQA and H64-MQA, as well as the CLA model H128-MQA-CLA2. In our initial design space exploration, our results for these models indicated that CLA makes it possible to shrink the KV cache footprint of an MQA model with $d_{\text{head}} = 128$ by a factor of $2\times$ while incurring only a small (0.06 point) degradation in perplexity, or to create a model with the same KV cache footprint as an MQA model with $d_{\text{head}} = 64$ while enjoying a substantial (0.21 point) improvement in perplexity. We wanted to verify that this qualitative pattern continues to hold when all models are trained with well-tuned learning rates.

Learning Rate Tuning Strategy. For each of our three model configurations, we swept the learning rate upwards from an initial value of 3×10^{-4} in multiplicative increments of $1.5\times$. We ended our sweep for each model at the point where validation perplexity stopped improving. We treat the learning rate which achieved the lowest validation perplexity for each model as an approximation of that model's optimal learning rate.

Results. We found an optimal learning rate of LR $= 1.5 \times 10^{-3}$ for our H128-MQA baseline, and a higher optimal learning rate of LR $= 2.25 \times 10^{-3}$ for both our H64-MQA baseline and our H128-MQA-CLA2 model.

The validation perplexity results from our 1B-scale learning rate tuning experiments can be found in Table 3. When comparing all three models at their best learning rates, we found that the qualitative result from our design space exploration continues to hold: our CLA2 model incurs only a small (0.04 point) validation perplexity degradation relative to our $d_{\text{head}} = 128$ baseline while enjoying a $2\times$ smaller KV cache footprint, and achieves a substantial (0.31 point) validation perplexity improvement compared to our $d_{\text{head}} = 64$ baseline while using the same amount of KV cache memory.

To further validate our results, we also evaluate our three learning-rate-tuned 1B-scale models under EleutherAI's LM Eval Harness [Gao et al., 2023] on Wikitext [Merity et al., 2016] perplexity and seven standard downstream benchmarks. We report the results of these evaluation in tables 3 and 4. On Wikitext perplexity, we observe a similar pattern as with validation perplexity, with our tuned CLA2 model achieving approximately the same (0.01 points better) perplexity as our $d_{\text{head}} = 128$ baseline, and substantially (0.71 points) better perplexity than our $d_{\text{head}} = 64$ baseline. On the

downstream evaluations, we found that none of our three models model consistently wins or loses across different benchmarks, and that all three models are consistently within 1–5 percentage points of each other.

Model	KV Bytes Per Token (16-bit)	Best LR	Validation Perplexity	Wikitext Perplexity
H128-MQA	10240	1.5×10^{-3}	**12.39**	19.30
H128-MQA-CLA2	5120	2.25×10^{-3}	12.43	**19.29**
H64-MQA	5120	2.25×10^{-3}	12.74	20.00

Table 3: Perplexity results from learning rate tuning experiments at 1B-parameter scale.

Model (Best LR)	Hellaswag	PIQA	WG	SciQ	OBQA	BoolQ	ARC-E
H128-MQA	**36.24**	69.15	**52.96**	**82.9**	19.0	**57.40**	**55.43**
H128-MQA-CLA2	36.01	69.15	51.93	82.6	**21.4**	53.21	53.87
H64-MQA	35.22	**69.21**	50.75	78.5	19.4	55.81	51.68

Table 4: Downstream benchmark results for 1B-scale models with tuned learning rates. The columns "WG" and "OBQA" denote "WinoGrande" and "OpenBookQA", respectively.

3.3 Experiments at 3B-Parameter Scale

To determine how CLA performs when applied to larger models, we trained a collection of models at the 3B-parameter scale both with and without CLA. We trained each of our 3B-scale model from scratch on 100B tokens from our SlimPajama corpus. The common architectural hyperparameters for our 3B-scale models can be found in Table 2.

Experiments at Head Dimension $d_{\text{head}} = 128$. We initially ran experiments to compare three 3B-scale models analogous to the models we selected for our learning rate tuning experiments at the 1B-parameter scale. Specifically, we compared a model using MQA-CLA2 and $d_{\text{head}} = 128$ to an MQA model with the same head dimension (and hence $2\times$ the KV cache footprint), and to an MQA model with a head dimension of $d_{\text{head}} = 64$ (and hence the same KV cache footprint). Based on our 1B-scale experiments, we expected that our MQA-CLA2 and MQA models with $d_{\text{head}} = 128$ would achieve similar perplexities to each other, and that both would outperform the $d_{\text{head}} = 64$ model.

We tuned the learning rates for these models according to the same learning rate tuning protocol we used at the 1B-parameter scale. After tuning the learning rates for each model, we observed a result different than we had expected: at 3B scale, our MQA-CLA2 model achieves substantially better perplexities than both our $d_{\text{head}} = 128$ and $d_{\text{head}} = 64$ MQA baselines. Moreover, our $d_{\text{head}} = 64$ MQA baseline model achieves better perplexities than our tuned $d_{\text{head}} = 128$ MQA baseline, despite having only $1/2$ as much KV cache capacity. We report the optimal learning rates and perplexities for these three models in Table 5.

As with our 1B-scale learning rate tuning experiments, we evaluate these models on downstream benchmarks. We report the results of these evaluations in Table 6. As with our 1B-scale experiments, we do not find that any model consistently wins or loses in these downstream evaluations.

Model	KV Bytes Per Token (16-bit)	Best LR	Validation Perplexity	Wikitext Perplexity
H128-MQA	16384	6.75×10^{-4}	9.52	13.63
H128-MQA-CLA2	8192	2.25×10^{-3}	**9.34**	**13.25**
H64-MQA	8192	1.00×10^{-3}	9.48	13.49

Table 5: Optimal learning rate and perplexity results for our first set of 3B-scale experiments.

Experiments at Head Dimension $d_{\text{head}} = 64$. Because in our initial 3B-scale experiments we found that our $d_{\text{head}} = 64$ MQA model represents a stronger baseline than our $d_{\text{head}} = 128$ MQA model, we ran a second set of experiments with adjusted head sizes. Specifically, we chose to compare

Model (Best LR)	Hellaswag	PIQA	WG	SciQ	OBQA	BoolQ	ARC-E
H128-MQA	45.73	73.07	60.46	88.1	25.4	59.30	**64.90**
H128-MQA-CLA2	**47.12**	**74.32**	**60.69**	**89.2**	25.2	58.62	64.73
H64-MQA	46.42	74.05	57.85	88.1	**25.6**	**59.88**	65.57

Table 6: Downstream evaluation results for our first set of 3B-scale experiments.

an MQA-CLA2 model with $d_{\text{head}} = 64$ to a plain MQA model with $d_{\text{head}} = 64$, and to a plain MQA model with $d_{\text{head}} = 32$.

Due to logistical constraints we trained all models in this second set of 3B-scale experiments on a different training cluster using a different training software stack and data order. This included retraining a new version of our H64-MQA-CLA2 baseline in order to control for differences in the new training environment.

We trained all models in this second set of experiments with a learning rate of LR $= 10^{-3}$, which we had found to be the best learning rate for our $d_{\text{head}} = 64$ MQA baseline model in our first set of 3B-scale experiments. For our $d_{\text{head}} = 64$ MQA-CLA2 model and our $d_{\text{head}} = 32$ MQA baseline model, we also experimented with learning rates of LR $\in \{6.75 \times 10^{-4}, 1.5 \times 10^{-3}\}$, but found these achieved worse perplexities than our initial value of LR $= 10^{-3}$.

We report perplexity results for this second set of experiments in Table 7, and results for downstream benchmarks in Table 8. In the Wikitext perplexity results for this set of experiments, we find agreement with the pattern observed at the 1B scale. Our MQA-CLA2 model with $d_{\text{head}} = 64$ incurs only a small (0.05 point) degradation in perplexity compared to our $d_{\text{head}} = 64$ baseline while enjoying a $2\times$ smaller KV cache footprint, and achieves a substantial (0.35 point) improvement in perplexity compared to our $d_{\text{head}} = 32$ baseline while using the same amount of KV cache memory.

We also evaluate these three models on downstream benchmarks, and report the results in Table 8. As with our downstream benchmark evaluations for our other experiments, we find that all models perform similarly.

Model	KV Bytes Per Token (16-bit)	Best LR	Wikitext Perplexity
H64-MQA	8192	1.0×10^{-3}	**12.94**
H64-MQA-CLA2	4096	1.0×10^{-3}	12.99
H32-MQA	4096	1.0×10^{-3}	13.34

Table 7: Optimal learning rate and perplexity results for our second set of 3B-scale experiments.

Model (Best LR)	Hellaswag	PIQA	WG	SciQ	OBQA	BoolQ	ARC-E
H64-MQA	**47.34**	**74.54**	**60.46**	**88.9**	24.2	57.25	**66.92**
H64-MQA-CLA2	47.32	**74.54**	57.46	87.9	25.2	61.62	65.53
H32-MQA	46.05	73.83	60.06	88.6	**25.6**	**61.87**	65.24

Table 8: Downstream evaluation results for our second set of 3B-scale experiments.

4 Discussion & Future Work

Takeaways and Implications. In the regimes where we tested it, we find that MQA-CLA2 consistently achieves the lowest validation perplexity (within 0.01 points) for a given KV cache memory budget and model size. In our ablations, we find that using sharing factors greater than 2 (CLA3 and above) achieves slightly worse accuracy/memory tradeoffs than using CLA2 and varying the head dimension, although still Pareto-dominates the tradeoffs possible with plain MQA alone.

At both 1B and 3B scale, we find that for MQA models with typical head sizes of 64 and 128, applying CLA2 yields a $2\times$ KV cache reduction while incurring at worst a very modest (less than 1% change) degradation in perplexity, and in some cases improving perplexity. We recommend this recipe to practitioners as a conservative change to existing MQA architectures which delivers substantial memory overhead reductions with relatively little risk.

Future Work. One natural question that rises from any memory efficient LLM alternative is its efficiency improvement when serving through longer sequences and greater batching. We leave end-to-end inference efficiency evaluations of large, long-context models employing CLA as an interesting problem for future work. We suspect that the types of LLMs that will stand to gain the most are the ones which have extremely long sequences, such as models that have long term memory or use methods like Landmark Attention [Mohtashami and Jaggi, 2023] which render attention over long contexts more feasible.

5 Related Work

Transformer memory efficiency can refer to many potential objectives. It can refer to decreasing memory storage or bandwidth requirements, it can be targeted at training or inference, and finally it apply either within a single pass of the model or between passes. While this work targets decreasing the size of the inference KV cache that persists between passes, notable works such as Flash Attention [Dao et al., 2022, Dao, 2023] have decreased the memory bandwidth necessary for a single pass, and works like FlexGen [Sheng et al., 2023] achieved low memory storage during a single forward pass by partial offloading to disk. Here we discuss related work that improves the memory efficiency of attention, specifically the memory storage of the KV cache.

5.1 Decreasing KV cache size Post training

Much work has focused on decreasing the size of the KV cache for models that have already been trained.

KV cache compression As many works have tried to compress LLMs through pruning, quantization, and sparsity, (see Zhu et al. [2023] for a survey) a subset directly focus on the problem of KV cache compression. For quantization, KVQuant [Hooper et al., 2024] and Coupled Quantization [Zhang et al., 2024] perform targeted transformations of the keys and values along with non uniform encodings to compress the KV cache to one to two bits. Sparsifying the KV cache done by works such as H2O [Zhang et al., 2023] Scissorhands [Liu et al., 2023] and FastGen [Ge et al., 2024] only store a subset of the KV cache during generation. They do so by storing only tokens that are near to the generating token or important across the sequence, with the heuristic for importance varying between papers. Finally, Cachegen [Liu et al., 2024] directly compresses the KV cache with a tensor encoder.

5.2 Architectural Changes that decrease KV cache size

Most relevant to our work are methods that change the architecture of the model in order to decrease the size of the KV cache. These methods can roughly grouped into three categories: those that try to reduce the number of tokens attended to, those that replace softmax attention with another operation that requires less memory storage, and those that, like our work, decrease the unique KV values compared to each query.

Decreasing effective sequence length Models that decrease the effective sequence length of the model have existed almost as transformers themselves. Two notable early works are Transformer XL [Dai et al., 2019] and Sparse Attention [Child et al., 2019]. Both of them performed attention in local windows of smaller sizes instead of across the whole sequence, and differed in how they incorporated information from prior tokens. This line of work was used in many models of note such as GPT3, and is commonly known as sliding window attention. This line of work has been further developed using methods like Infini attention [Munkhdalai et al., 2024], which compresses prior tokens using a linear attention mechanism.

An alternative approach is to perform a lookup over prior tokens, such as in Landmark attention [Mohtashami and Jaggi, 2023] Memorizing transformers [Wu et al., 2022] or to do a lookup over an external datastore, which is commonly known as retrieval [Guu et al., 2020, Izacard et al., 2024, Borgeaud et al., 2022]. However, while these methods reduce computation, they do not reduce storage unless the KVs for lookup are offloaded from GPU memory.

Removing Softmax Attention Replacements to Softmax Attention are often referred to as SSMs or linear attention computations. Regardless of name, most replace the attention component with an alternative which has constant space complexity during token generation w.r.t. the number of tokens generated. This also reduces the time complexity of generation to be linear w.r.t. the number of tokens, instead of quadratic. Various methods differ in how they parameterize their state. Katharopoulos et al. [2020], Wang et al. [2020] proposed initial versions of linear attention. Recent work has focused on using data dependent mechanisms to improve the state, such as in GLA [Yang et al., 2024] Mamba[Gu and Dao, 2023] and RWKV v6 [Peng et al., 2024].

Groupings of Attention Most related are methods that use softmax attention but attempt to use a single KV pair for multiple queries. GQA [Ainslie et al., 2023a] and MQA [Shazeer, 2019] do this by grouping keys and values across heads.

Concurrent work tries other strategies to share values between layers. Deepseek-V2 [DeepSeek-AI, 2024] proposes Multi Latent Attention, which uses a low rank projection of the keys and values. You Only Cache Once [Sun et al., 2024] splits the model into two halves. The first half performs local attention and then generates a set of keys and values that are used for global attention across all layers in the second half.

6 Conclusion

Cross-Layer Attention is an effective method for reducing the KV cache memory storage footprint of transformer models by a factor of $2\times$ with roughly equal perplexity. Based on extensive experimental evaluation against well-tuned baselines at both the 1B- and 3B-parameter scales, we find that CLA advances the Pareto frontier for memory-efficient transformers.

References

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In H. Bouamor, J. Pino, and K. Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, Singapore, Dec. 2023a. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.298. URL `https://aclanthology.org/2023.emnlp-main.298`.

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023b.

J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, Apr. 2024. doi: 10.1145/3620665.3640366. URL `https://pytorch.org/assets/pytorch2-2.pdf`.

S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach. Gpt-neox-20b: An open-source autoregressive language model, 2022.

S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. De Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. Rae, E. Elsen, and L. Sifre. Improving language models by retrieving from trillions of tokens. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of

Proceedings of Machine Learning Research, pages 2206–2240. PMLR, 17–23 Jul 2022. URL https://proceedings.mlr.press/v162/borgeaud22a.html.

R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019. URL http://arxiv.org/abs/1904.10509.

A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel. Palm: Scaling language modeling with pathways, 2022.

Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. Le, and R. Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In A. Korhonen, D. Traum, and L. Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1285. URL https://aclanthology.org/P19-1285.

T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.

B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo. Attentionstore: Cost-effective attention reuse across multi-turn conversations in large language model serving, 2024.

L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao. Model tells you what to discard: Adaptive kv cache compression for llms, 2024.

Google. Context caching guide. https://ai.google.dev/gemini-api/docs/caching, 2024. Accessed: 2024-05-20.

A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.

K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. Realm: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization, 2024.

Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen. *GPipe: efficient training of giant neural networks using pipeline parallelism*. Curran Associates Inc., Red Hook, NY, USA, 2019.

G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave. Atlas: few-shot learning with retrieval augmented language models. *J. Mach. Learn. Res.*, 24(1), mar 2024. ISSN 1532-4435.

D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, J. Yang, J. Park, A. Heinecke, E. Georganas, S. Srinivasan, A. Kundu, M. Smelyanskiy, B. Kaul, and P. Dubey. A study of bfloat16 for deep learning training, 2019.

A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are rnns: fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang. Cachegen: Kv cache compression and streaming for fast language model serving, 2024.

Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time, 2023.

I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. URL `https://openreview.net/forum?id=Skq89Scxx`.

I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=Bkg6RiCqY7`.

S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models, 2016.

P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training, 2018.

A. Mohtashami and M. Jaggi. Random-access infinite context length for transformers. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 54567–54585. Curran Associates, Inc., 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/file/ab05dc8bf36a9f66edbff6992ec86f56-Paper-Conference.pdf`.

T. Munkhdalai, M. Faruqui, and S. Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention, 2024.

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.

B. Peng, D. Goldstein, Q. Anthony, A. Albalak, E. Alcaide, S. Biderman, E. Cheah, X. Du, T. Ferdinan, H. Hou, P. Kazienko, K. K. GV, J. Kocoń, B. Koptyra, S. Krishna, R. M. J. au2, N. Muennighoff, F. Obeid, A. Saito, G. Song, H. Tu, S. Woźniak, R. Zhang, B. Zhao, Q. Zhao, P. Zhou, J. Zhu, and R.-J. Zhu. Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence, 2024.

P. Ramachandran, B. Zoph, and Q. V. Le. Searching for activation functions, 2017.

N. Shazeer. Fast transformer decoding: One write-head is all you need, 2019.

N. Shazeer. Glu variants improve transformer, 2020.

Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang. Flexgen: high-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

D. Soboleva, F. Al-Khateeb, R. Myers, J. R. Steeves, J. Hestness, and N. Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. https://www.cerebras.net/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama, 2023. URL https://huggingface.co/datasets/cerebras/SlimPajama-627B.

J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, Q. Zhang, J. Wang, and F. Wei. You only cache once: Decoder-decoder architectures for language models, 2024.

H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

C. Wang, K. Cho, and J. Gu. Neural machine translation with byte-level subwords, 2019.

S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. *CoRR*, abs/2006.04768, 2020. URL https://arxiv.org/abs/2006.04768.

Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=TrjbxzRcnf-.

S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim. Gated linear attention transformers with hardware-efficient training, 2024.

T. Zhang, J. Yi, Z. Xu, and A. Shrivastava. Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization, 2024.

Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. A. Wang, and B. Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/6ceefa7b15572587b78ecfcebb2827f8-Paper-Conference.pdf.

X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang. A survey on model compression for large language models, 2023.

A Learning rate sweeps

Here we present the results of our learning rate sweeps at the 1B and 3B parameter scales:

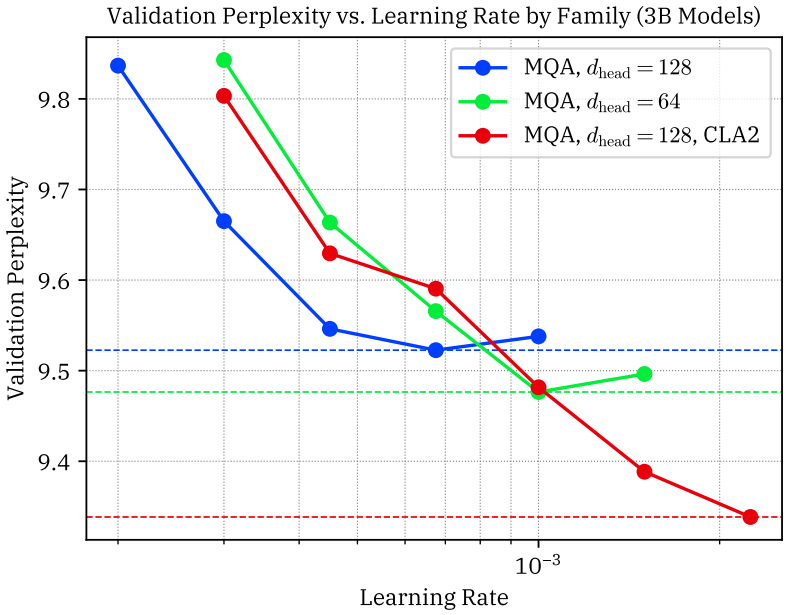

Figure 4: 3B Learning Rate Sweep

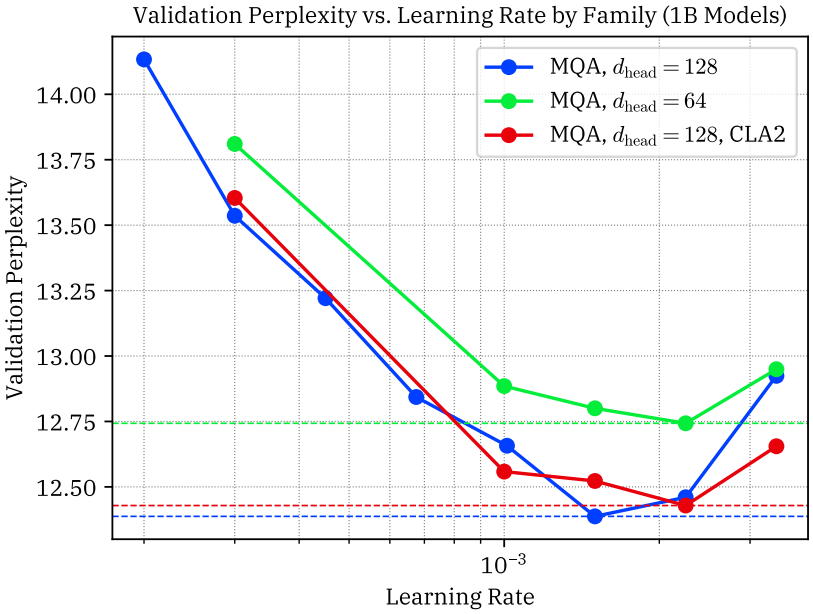

Figure 5: 1B Learning Rate Sweep